TOBB University of Economics and Technology

Department of Computer Engineering

Department of Artificial Intelligence Engineering

AI Health System for Doctors and Patients

Final Report

Abdullah Çöplü

Abdurrahman Doğru

Ece Derya

Zeynep Ergül

Table of Content

1. Introduction 3
2. System Overview & Final Architecture 3
3. Implementation Details 5
    3.1 System Implementation 5
    3.2 Model Architectures Used 7
        3.2.1 Resnet50 7
        3.2.2 DenseNet121 8
        3.2.3 EfficientNet-B4 8
        3.2.4 Vision Transformer (ViT-B/16) 9
    3.3 Training Workflow 10
4. Test Results 11
    4.1. Summary of Executed Test Cases 11
    4.2. AI Model Evaluation Results for Integrated Diagnostic Tasks 13
    4.3. Additional Experimental Result 20
5. Impact of Engineering Solutions 20
    5.1. Global Impact 20
    5.2. Economic Impact 21
    5.3. Environmental Impact 21
    5.4. Societal Impact 22
6. Contemporary Issues Discussion 22
7. Tools and Technologies Used 24
8. Use of Resources and Related Work 25
9. Engineering Standards & Design Principles 27
10. New Tools & Technologies Used 30
11. Discussion, Evaluation & Future Work 31
12. Conclusion 32
13. References 34

1. Introduction

Artificial intelligence is increasingly being integrated into healthcare systems to support clinical decision-making, improve diagnostic efficiency, and expand access to medical guidance. However, deploying AI in medical contexts requires not only strong predictive performance but also careful consideration of safety, interpretability, and ethical responsibility.

In this project, we present an AI-assisted medical decision-support platform designed for both doctors and patients. The system provides a unified web-based environment that combines medical image analysis and natural language-based symptom evaluation. Rather than acting as an autonomous diagnostic tool, the platform positions AI as a supportive component within human-centered workflows.

From the physician perspective, the system enables case creation, medical image upload, AI-assisted result review, annotation, and override mechanisms. From the patient perspective, it offers a simplified conversational interface that delivers moderated, understandable responses with enforced disclaimers. This role-aware design ensures that outputs are tailored according to the needs and responsibilities of each user group.

The system is implemented using a modular full-stack architecture, including a React-based frontend, a FastAPI backend, AI inference services, and a persistent data layer. This structure follows established low-level design principles by transforming high-level system components into well-defined, implementable modules,enabling maintainability, scalability, and controlled workflow execution.

Overall, the project demonstrates how artificial intelligence can be integrated into healthcare systems in a way that is technically functional, safe, and aligned with real-world clinical and patient-facing requirements.

2. System Overview & Final Architecture

System Overview

The AI Health System for Doctors and Patients is a web-based, AI-assisted medical decision-support platform developed for two primary user groups: doctors and patients. The system integrates medical image analysis and symptom-based language analysis within a single platform while enforcing role-specific behavior, safety constraints, and ethical limitations. Its primary purpose is to support clinical workflows, improve access to early diagnostic assistance, and provide structured guidance without replacing licensed medical professionals.

From the physician perspective, the platform supports case creation, medical image upload, AI-assisted result review, confidence interpretation, annotation, feedback submission, and override workflows. Accordingly, AI outputs are positioned as supportive clinical artifacts rather than autonomous decisions. From the patient perspective, the system provides a simplified natural-language symptom interface that returns moderated,

understandable, and disclaimer-covered responses. This design ensures that patient-facing outputs remain controlled and do not expose users to raw, overly technical, or potentially anxiety-inducing content.

The implemented platform combines three principal domains: the application layer, the AI inference layer, and the model development layer. The application layer governs authentication, authorization, persistence, and interaction workflows. The AI layer provides both image-based and text-based analysis capabilities. The model development layer supports the training and preparation of deployable AI models used by the system. Together, these domains establish an integrated and extensible medical support environment.

Final Architecture

The final architecture follows a layered and modular structure. At the presentation layer, a React and TypeScript based frontend provides distinct interfaces for doctors, patients, and administrators. Doctor-facing views emphasize case management, image upload, and AI-supported review, whereas patient-facing views focus on symptom submission, chat continuity, and safe response presentation.

At the application layer, the backend is implemented using FastAPI and follows a controller-service-repository organization. Controllers process API requests, services implement business logic, and repositories manage persistence. This architectural approach enables clearer separation of responsibilities, stronger maintainability, and more controlled execution of safety-sensitive workflows.

The AI layer consists of two main branches. The first is the vision inference subsystem, which supports multiple task-specific medical imaging models through a registry-based design. These models are used for tasks such as chest X-ray analysis, brain MRI classification, retinal OCT analysis, skin lesion classification, fracture detection, COVID radiography analysis, breast ultrasound analysis, and NIH chest X-ray interpretation. The second branch is the symptom analysis subsystem, which relies on an external HPC-hosted language model service based on MedGemma-27B-IT and is supported by Turkish-English translation(Sellergren et al., 2025; Tiedemann, 2020) models for multilingual interaction.

At the persistence layer, the system stores users, sessions, settings, cases, uploaded images, AI results, symptom reports, chat sessions, chat messages, feedback, annotations, overrides, and audit records. This persistence structure supports both operational continuity and traceability of user actions and AI-supported decisions.

Finally, the platform incorporates a monitoring and observability layer. Health checks, logging, metrics collection, and audit mechanisms are integrated in order to improve reliability, operational visibility, and post-event traceability. As a result, the final

architecture reflects the project's main engineering priorities: modularity, safety, traceability, extensibility, and responsible integration of artificial intelligence into healthcare support workflows.

3. Implementation Details

3.1 System Implementation

The implementation of the AI Health System for Doctors and Patients was carried out as a full-stack, role-aware software platform integrating web technologies, backend services, database persistence, and AI inference components. The implementation process focused not only on functional delivery, but also on safe workflow control, modularity, and traceability, which were particularly important given the medical decision-support nature of the project.

Frontend Implementation
At the frontend level, the system was implemented as a browser-based client application using React and TypeScript. The user interface was designed around role-specific workflows for doctors, patients, and administrators. Doctor-facing pages were implemented to support case creation, medical image upload, AI result visualization, annotation, feedback submission, and override actions. Patient-facing pages were implemented to support symptom submission, session-based conversational interaction, and simplified response presentation with visible disclaimers. The frontend also includes language-aware interface behavior and secure API communication with the backend.

Backend Implementation
At the backend level, the system was implemented using FastAPI. The backend follows a controller-service-repository organization, allowing HTTP request handling, business logic, and persistence logic to remain clearly separated. Authentication and authorization were implemented using JWT-based access control together with session-aware validation. Role-based access restrictions were enforced across protected endpoints so that patient, doctor, and administrator functions remained isolated according to system rules.

Medical Image Analysis Workflow
The medical image analysis workflow was implemented through a dedicated vision service capable of supporting multiple task-specific models. Instead of relying on a single generic model, the system uses a registry-based approach in which each model is associated with a specific diagnostic task, architecture, preprocessing pipeline, and output structure. This allows the same backend service to manage different image-analysis workflows while preserving model-specific behavior. Uploaded images are validated, associated with cases, passed through inference, and then stored together with structured prediction outputs for later review and traceability.

Symptom Analysis Workflow

The symptom analysis workflow was implemented as a controlled conversational pipeline. User symptom input is submitted through the frontend and processed by the backend only after consent requirements are satisfied where applicable. The backend then sends the structured request to an external HPC-hosted language model service based on MedGemma-27B-IT[1]. Since the patient-facing workflow also needed to support Turkish interaction, the system was integrated with Helsinki-NLP translation models for Turkish-to-English[2] and English-to-Turkish[3] conversion. In this way, Turkish symptom input could be translated into English before LLM processing, and the generated response could then be translated back into Turkish for presentation to the user.

The returned result is not exposed directly. Instead, the backend applies role-aware presentation logic, disclaimer enforcement, and safe fallback behavior before delivering a response to the user. This design ensures that language-model integration remains useful while still being bounded by application-level safety controls.

In addition to the immediate symptom text entered by the user, the symptom-analysis workflow was designed to incorporate available patient-specific contextual information such as demographic attributes, relevant medical history, chronic diseases, medications, allergies, and health-related preferences when such data had been provided through the system. This allowed the language-model-based analysis to produce more context-aware responses rather than treating each symptom statement as an isolated input. In this way, the conversational pipeline was not limited to surface-level text generation, but was structured to support more personalized and medically contextualized guidance under application-level safety controls.

Persistence and Traceability

A persistent data layer was implemented to support system continuity and traceability. The platform stores user records, active sessions, user settings, medical cases, medical images, AI inference results, symptom reports, chat sessions, chat messages, doctor annotations, feedback records, overrides, and audit events. This persistence model enables both operational functionality and retrospective examination of system behavior, which is essential in a healthcare-related platform. Auditability was treated as an implementation concern rather than an afterthought.

Monitoring and Observability

The implementation also includes operational and observability features. Logging, health checks, metrics exposure, and audit recording were integrated into the backend. Monitoring-related components were included in the deployment environment to provide visibility into runtime behavior, service status, and request-level characteristics. This was

[1] https://huggingface.co/google/medgemma-27b-it
[2] https://huggingface.co/Helsinki-NLP/opus-mt-tr-en
[3] https://huggingface.co/Helsinki-NLP/opus-tatoeba-en-tr

especially valuable for detecting latency issues, external service failures, and unexpected behavior during testing and integration.

Deployment

From a deployment perspective, the project supports both local development execution and container-based orchestration. The backend was containerized and configured together with supporting services such as the database, cache, and monitoring tools. This allowed the project to be tested and run in a more controlled environment while preserving separation between the application, persistence, and observability layers.

Implementation Outcome

Overall, the implementation phase transformed the system from a design-level concept into a working AI-assisted healthcare support platform. The final result was not merely a prototype interface, but an integrated software system combining role-based web interaction, backend-controlled safety logic, multi-model image inference, language-model-based symptom analysis, persistence, and operational monitoring within a single coherent architecture.

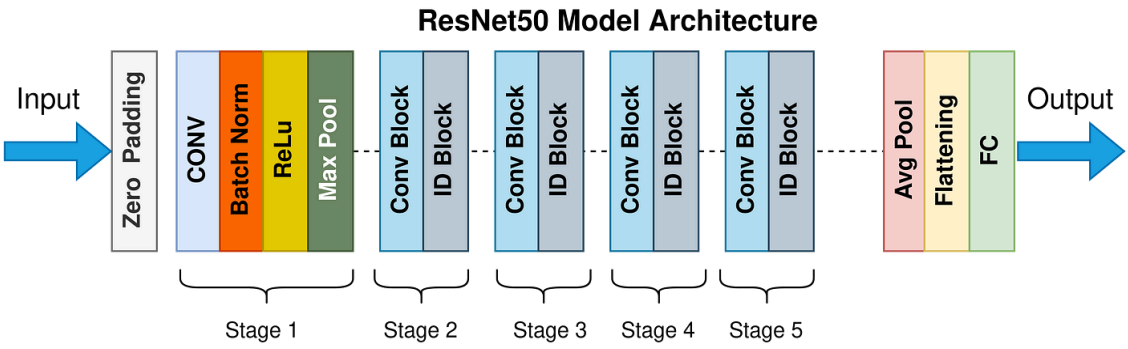
3.2 Model Architectures Used

To support different medical image analysis tasks, the system incorporated multiple deep learning architectures rather than relying on a single model family. This decision was made because different diagnostic tasks may benefit from different representational strengths, such as local feature extraction, feature reuse, computational efficiency, or global contextual understanding. The principal architecture families used in the project were ResNet50[4], DenseNet121[5], EfficientNet-B4[6], and Vision Transformer[7] (ViT-B/16) (He et al., 2016; Huang et al., 2017; Tan and Le, 2019; Dosovitskiy et al., 2021).

3.2.1 Resnet50

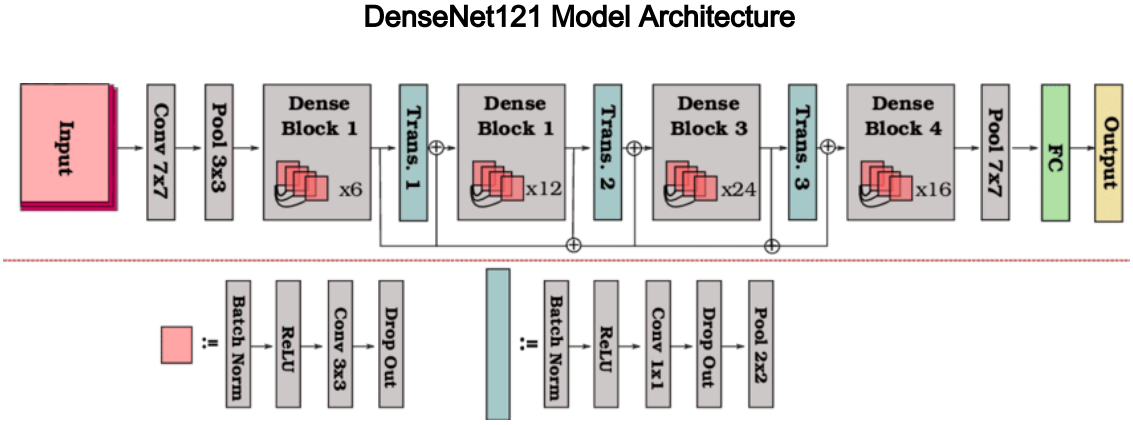
ResNet50 is a convolutional neural network architecture based on residual learning. Its main advantage is the use of skip connections, which help deeper networks train more effectively by reducing vanishing-gradient problems. In the context of this project, ResNet50 served as an important baseline architecture for medical image classification tasks, particularly in cases where robust hierarchical feature extraction was required.

[4] https://huggingface.co/microsoft/resnet-50
[5] https://huggingface.co/timm/densenet121.tv_in1k
[6] https://huggingface.co/google/efficientnet-b4
[7] https://huggingface.co/google/vit-base-patch16-224



3.2.2 DenseNet121

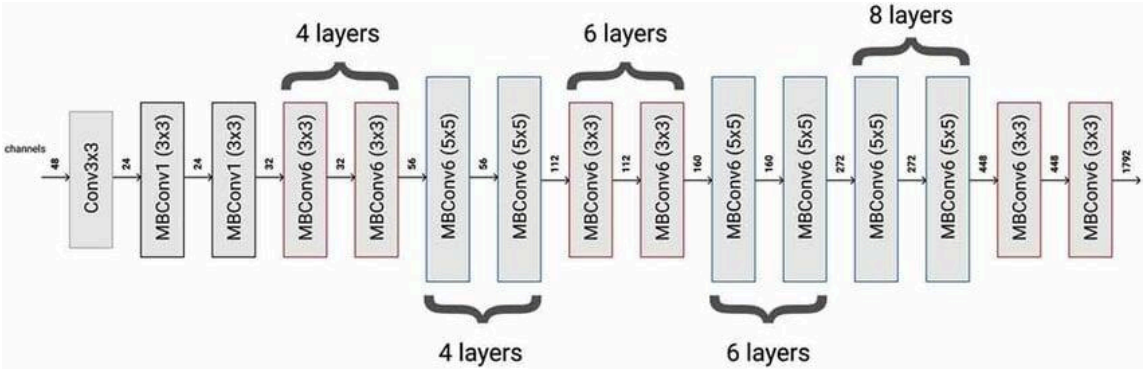
DenseNet121 is another convolutional architecture in which each layer receives feature maps from all preceding layers. This dense connectivity improves feature reuse and strengthens information flow throughout the network. For medical imaging tasks, DenseNet121 is especially relevant because subtle diagnostic patterns may benefit from repeated access to low-level and high-level features. In this project, DenseNet121 was considered a strong architecture for classification-oriented image analysis.



3.2.3 EfficientNet-B4

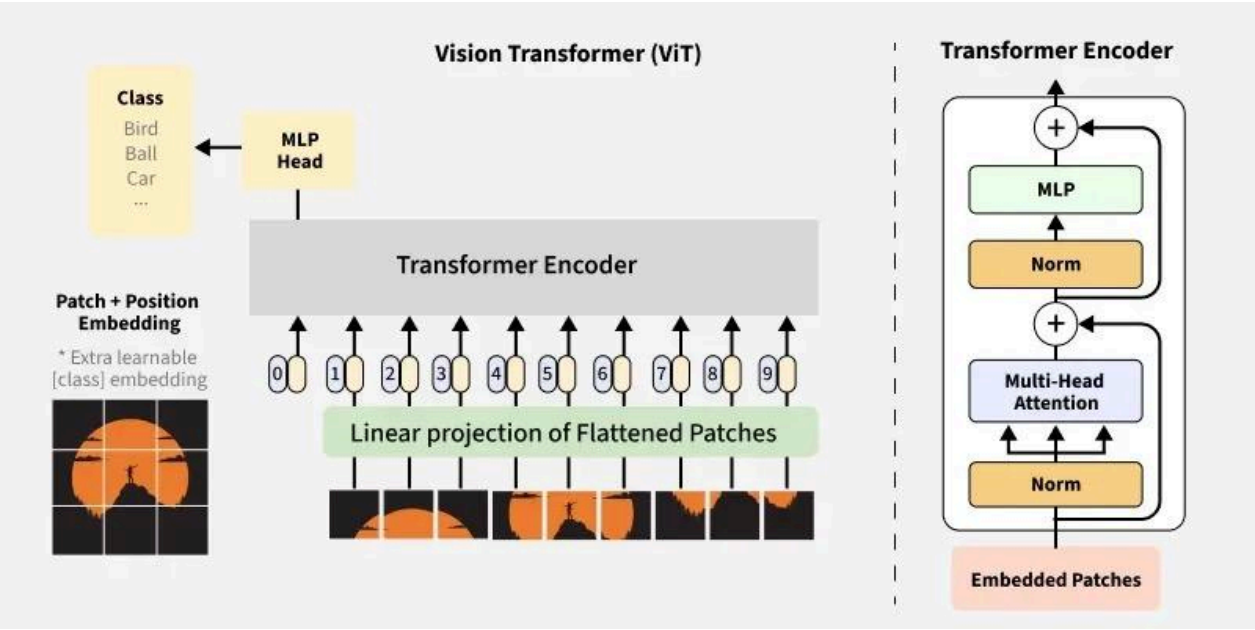
EfficientNet-B4 belongs to the EfficientNet family, which scales network depth, width, and input resolution in a balanced manner. This makes it attractive for medical AI applications where predictive performance must be achieved without excessive computational overhead. Within the project, EfficientNet-B4 represented an efficient convolutional alternative for image analysis tasks requiring strong performance with a comparatively optimized model structure.

EfficientNet-B4 Model Architecture



3.2.4 Vision Transformer (ViT-B/16)

Vision Transformer applies the transformer paradigm to image data by dividing an image into patches and processing them as a sequence. Unlike traditional convolutional networks, ViT is particularly effective at modeling global contextual relationships across the image. In this project, ViT-based models were used for tasks where capturing broader structural patterns was important, making them a valuable complement to CNN-based architectures.



These architectures were integrated into the registry-based vision service according to task requirements, model availability, and deployment constraints, enabling the system to support multiple diagnostic workflows within a unified implementation framework.

3.3 Training Workflow

Data Preparation and Preprocessing

The training workflow was designed to support the development of task-specific medical image models that could later be integrated into the platform's registry-based inference system. Rather than treating model training as an isolated experimental process, the workflow was structured to produce deployable model artifacts suitable for backend integration and real-time inference.
The workflow began with dataset collection and preparation. Publicly available medical imaging datasets were selected according to the diagnostic tasks targeted in the project. After dataset selection, the images were prepared through a task-aware preprocessing pipeline. All input images were resized to 224 × 224, which provided a common resolution for the selected architectures. For EfficientNet-B4, bicubic interpolation was used during resizing, while the remaining models used standard resizing.

During training, data augmentation was applied in order to improve generalization. The common augmentation pipeline included random horizontal flipping with probability 0.5 and color jittering with brightness and contrast adjustments. For X-ray-based tasks, additional random rotation was applied in order to simulate imaging angle variation. After augmentation, the images were converted into tensors and normalized. Vision Transformer based models used normalization with mean and standard deviation values of [0.5, 0.5, 0.5], whereas convolution-based models used ImageNet normalization values of [0.485, 0.456, 0.406] for the mean and [0.229, 0.224, 0.225] for the standard deviation`.

Dataset Partitioning

For dataset partitioning, the workflow was adapted according to the structure of each dataset. In some tasks, predefined training, validation, and test splits were provided through structured annotation files, while in others the split structure was already organized at the directory level. This flexible design allowed the training pipeline to accommodate different dataset formats without changing the overall training logic. In addition, robust image loading was enabled in cases where image files could be partially corrupted, improving stability during both training and evaluation.

Model Training and Optimization

Following data preparation, task-specific models were trained using architectures selected according to problem characteristics and computational considerations. Convolution-based architectures ResNet50, DenseNet121, and EfficientNet-B4 were used as strong baselines for image classification tasks, while Vision Transformer based model was incorporated for tasks that benefited from broader contextual representation. During training, the workflow used architecture-specific hyperparameter settings, including customized learning rates, batch sizes, weight decay values, and scheduler strategies. For example, transformer-based models used lower learning rates and cosine

annealing schedules, whereas convolution-based models generally used either step-based scheduling or fixed learning rate strategies depending on the model family. The optimization process was carried out using AdamW, while the loss function was selected according to the classification problem. Binary tasks used BCEWithLogitsLoss, whereas multi-class tasks used CrossEntropyLoss. During each training run, model performance was monitored on the validation set, and the best-performing checkpoint was selected according to validation F1-score. This ensured that model selection was based not only on training convergence, but also on generalization quality.

Evaluation

After training, the selected checkpoints were evaluated on an unseen test set using task-appropriate metrics such as accuracy, precision, recall and F1-score. These trained model weights were then exported and prepared for integration into the backend vision service, where each model could be registered together with its preprocessing configuration, architecture family, and target task.

In this way, the training workflow did not end at model evaluation, but continued through deployment-oriented preparation. This ensured that the AI components used in the final platform were not only experimentally developed, but also structurally aligned with the implementation architecture of the overall system.

4. Test Results

Testing activities were conducted in accordance with the prepared Test Plan Report and covered unit, integration, system, scenario, performance, and model-quality validation. The purpose of testing was to verify not only the correctness of individual software components, but also the reliability, safety, and traceability of complete doctor- and patient-facing workflows. At the end of the project, all planned test activities were completed.

4.1. Summary of Executed Test Cases

Test ID	Description	Input(s)	Expected Output	Result
TC-01	Verify user creation and retrieval in DB	Valid User Model data	User record successfully created and fetched	**PASSED**
TC-02	Fetch user by email address from DB	Existing user's email string	Returns the correct User object	**PASSED**
TC-03	End-to-end patient symptom workflow	Patient Auth Token + Symptom details	Successful analysis and DB record creation	**PASSED**
TC-04	Enforce auth on symptom endpoints	Request without Bearer Token	*401/403* HTTP Status (Unauthorized/Forbidden)	**PASSED**
TC-05	Check consent requirement for symptoms	Valid Token, *Consent=False*, Symptom Text	*403* HTTP Status (Consent required)	**PASSED**

TC-06	Complete full symptom analysis flow	Valid Token, *Consent=True*, Symptom Text	*200 OK*, valid AI prediction and risk level	**PASSED**
TC-07	Verify password hashing mechanism	Raw string password	Hashed string (not matching raw input)	**PASSED**
TC-08	JWT access token generation	User credentials (ID, Role)	Valid, decodable JWT token string	**PASSED**
TC-09	Retrieve existing chat session	Existing session_id	Returns the correct chat session history	**PASSED**
TC-10	Create a new chat session automatically	No session_id + Initial message text	Generates and returns a new session_id	**PASSED**
TC-11	Validate user creation schema (Positive)	Valid email and password structure	Schema validation passes	**PASSED**
TC-12	Validate user creation schema (Negative)	Invalid email format (e.g., "test@com")	Schema validation throws ValidationError	**PASSED**
TC-13	Validate symptom submit request schema	Valid text string	Schema validation passes	**PASSED**
TC-14	Block empty symptom submit requests	Empty string ("")	Schema validation throws ValidationError	**PASSED**
TC-15	Verify role-based UI visibility for Doctor and Patient	Doctor login, Patient login	Correct role-specific screens/features are visible	**PASSED**
TC-16	Verify patient result screen shows disclaimer and non-technical safe output	Valid symptom submission	Valid symptom submission Disclaimer is visible; output is understandable and non-technical	**PASSED**
TC-17	Verify doctor can view previous patient/chat interaction history	Valid doctor session + existing history/session	Prior interactions are displayed correctly	**PASSED**
TC-18	Verify Turkish/English language presentation in UI	Switch language to TR and EN	User-facing texts change correctly	**PASSED**
TC-19	Verify doctor feedback / annotation / override workflow	Valid doctor action payload	Feedback/annotation/override is saved successfully	**PASSED**
TC-20	Verify doctor medical image upload and AI result viewing	Valid doctor token + supported medical image	Upload succeeds and doctor can view AI result	**PASSED**
TC-21	Verify logging/monitoring record creation for critical actions	Trigger symptom analysis / doctor action	Corresponding log/monitoring record is created	**PASSED**
TC-22	Verify response-time and concurrency requirements	Timed requests / concurrent users	Meets defined latency and load expectations	**PASSED**
TC-23	Verify model evaluation metrics for all models	Test dataset + trained model	Accuracy and related metrics are produced successfully	**PASSED**
TC-24	Verify FNR and FPR calculation for critical classes	Validation predictions + labels	FNR and FPR values are computed correctly	**PASSED**

The completed tests show that the platform satisfies its core functional, safety-oriented, and workflow-level requirements. Authentication, authorization, consent handling, symptom submission, safe response generation, doctor-side image analysis, audit

logging, localization, and system integration behaviors were all verified successfully. Based on the final test execution status, 24 out of 24 planned test cases were completed successfully, with no unresolved critical or blocking defect remaining at project closure.

4.2. AI Model Evaluation Results for Integrated Diagnostic Tasks

In addition to software-oriented verification, model-level evaluation was conducted for all seven diagnostic tasks integrated into the final platform. Since the system includes binary and multiclass prediction problems across different medical imaging modalities, model performance was assessed in a task-aware manner using accuracy, precision, recall and F1-score. For each task, multiple model architectures were compared during training, and the best-performing model was selected for deployment according to both predictive performance and suitability for system integration.

Task	Model	Accuracy	Precision	Recall	F1-score
Breast Ultrasound	ViT-B/16	0.9043	0.9216	0.7705	0.8393
Breast Ultrasound	DenseNet121	0.8670	0.8000	0.7869	0.7934
Breast Ultrasound	ResNet50	0.8617	0.7778	0.8033	0.7903
Breast Ultrasound	EfficientNet-B4	0.8457	0.7857	0.7213	0.7521
COVID Radiography	ViT-B/16	0.9854	0.9882	0.9882	0.9882
COVID Radiography	DenseNet121	0.9835	0.9885	0.9866	0.9876
COVID Radiography	ResNet50	0.9825	0.9888	0.9852	0.9868
COVID Radiography	EfficientNet-B4	0.9764	0.9811	0.9801	0.9805
Fracture X-Ray	ViT-B/16	0.9921	1.0000	0.9832	0.9915
Fracture X-Ray	ResNet50	0.9921	1.0000	0.9832	0.9915
Fracture X-Ray	EfficientNet-B4	0.9901	0.9957	0.9832	0.9894
Fracture X-Ray	DenseNet121	0.9862	1.0000	0.9706	0.9851

Retinal OCT	ViT-B/16	0.9979	0.9979	0.9979	0.9979
Retinal OCT	DenseNet121	0.9812	0.9815	0.9808	0.9811
Retinal OCT	EfficientNet-B4	0.9768	0.9774	0.9761	0.9767
Retinal OCT	ResNet50	0.9694	0.9702	0.9687	0.9692
Skin Cancer	ViT-B/16	0.8051	0.7346	0.7315	0.7312
Skin Cancer	DenseNet121	0.7486	0.6925	0.6714	0.6813
Skin Cancer	EfficientNet-B4	0.7219	0.6612	0.6428	0.6514
Skin Cancer	ResNet50	0.6735	0.6038	0.5879	0.5953
Brain MRI Classification	ViT-B/16	0.977	0.978	0.977	0.977
Brain MRI Classification	DenseNet121	0.976	0.977	0.976	0.977
Brain MRI Classification	EfficientNet-B4	0.972	0.973	0.973	0.973
Brain MRI Classification	ResNet50	0.9710	0.972	0.971	0.9711
NIH Chest X-ray	DenseNet121	0.6835	0.6418	0.6706	0.66
NIH Chest X-ray	EfficientNet-B4	0.6521	0.6214	0.6355	0.6283
NIH Chest X-ray	ViT-B/16	0.6244	0.5980	0.6112	0.6045
NIH Chest X-ray	ResNet50	0.5932	0.5722	0.5840	0.5780

The results indicate that the selected model portfolio achieved strong performance across the integrated diagnostic tasks, while also showing that performance varied depending on imaging modality, class structure, and dataset difficulty. Such variation is expected in medical AI, since different tasks involve different levels of visual similarity, class imbalance, and diagnostic complexity. Therefore, the evaluation supports the use of task-specific model selection rather than relying on a single general-purpose architecture.

To complement the aggregate performance metrics, confusion-matrix-based error analysis was presented. These visualizations provide a more interpretable view of model behavior by showing where predictions are correct, where clinically relevant confusion occurs, and which classes remain more difficult to distinguish. In the context of medical

decision-support, such examples are valuable because they reveal class-level strengths and weaknesses that may not be fully visible through scalar metrics alone.

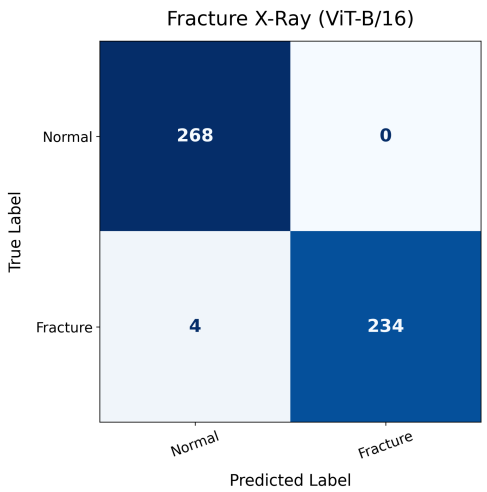


Figure 4.1. Confusion matrix of the best-performing Fracture X-Ray classification model (ViT-B/16).

Figure 4.1 demonstrates that the model separates fracture and normal cases with very high reliability. The small number of misclassified fracture samples suggests strong binary discrimination performance, while also indicating that a limited false-negative risk still remains and should be considered carefully in medical support settings.

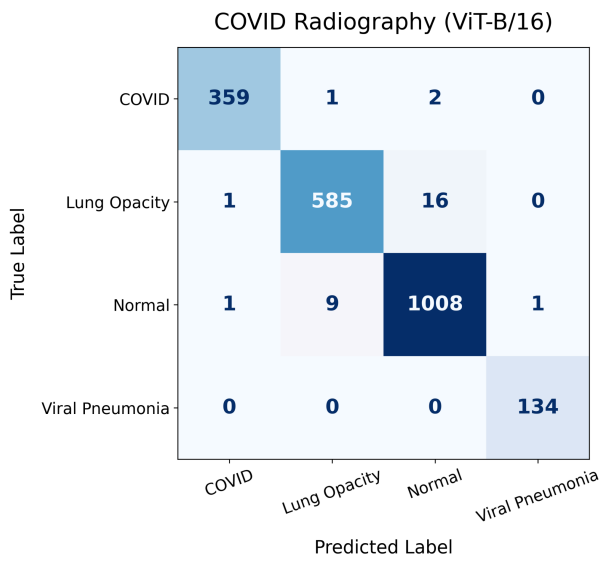


Figure 4.2. Confusion matrix of the best-performing COVID Radiography classification model (ViT-B/16).

Figure 4.2 indicates that the selected model achieves highly consistent separation across COVID, Lung Opacity, Normal, and Viral Pneumonia classes. The small number of off-diagonal predictions suggests that the model handles inter-class similarity effectively, although a few clinically related pulmonary findings remain harder to separate perfectly.

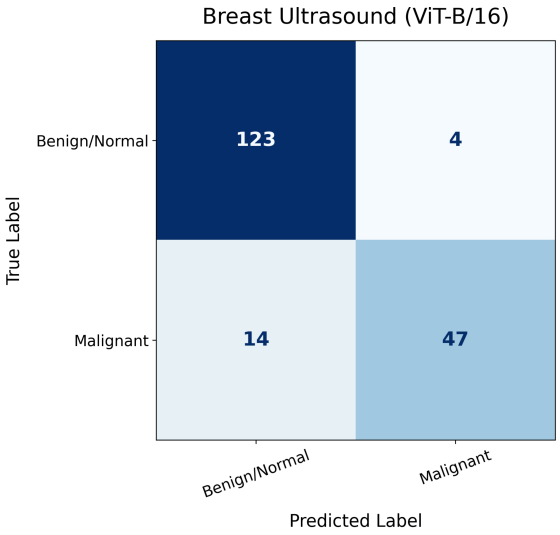


Figure 4.3. Confusion matrix of the best-performing Breast Ultrasound classification model (ViT-B/16).

Figure 4.3 shows that the selected model distinguishes benign/normal and malignant cases with strong overall performance, while still producing a limited number of false negatives and false positives. This result is important because breast ultrasound is a clinically sensitive binary task in which classification errors may directly affect diagnostic interpretation.

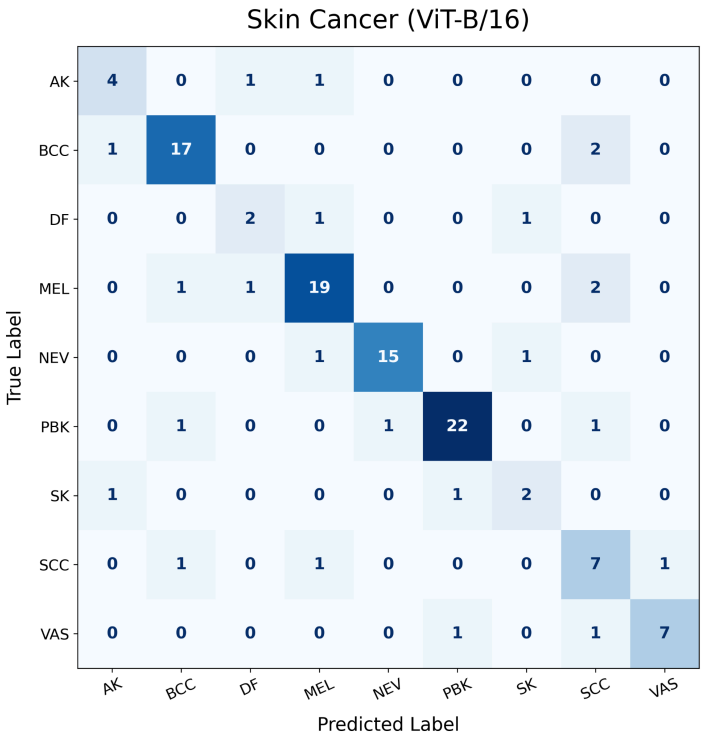


Figure 4.4. Confusion matrix of the best-performing Skin Cancer classification model (ViT-B/16).

Figure 4.4 shows that the skin lesion classification task remains more challenging than the other imaging tasks because several lesion categories share visually similar characteristics. Although the selected model demonstrates comparatively strong overall performance, the confusion matrix indicates that some diagnostically related classes remain harder to separate, highlighting the need for careful interpretation and continued model improvement in dermatology-oriented use cases.

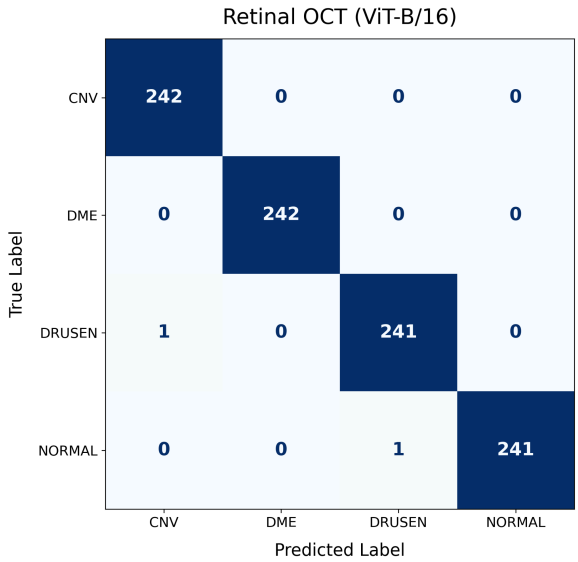


Figure 4.5. Confusion matrix of the best-performing Retinal OCT classification model (ViT-Base).

Figure 4.5 shows that the selected model distinguishes CNV, DME, DRUSEN, and NORMAL classes with near-perfect consistency. The very limited off-diagonal errors indicate that the model captures class-specific retinal patterns effectively and provides highly reliable separation across the evaluated categories.

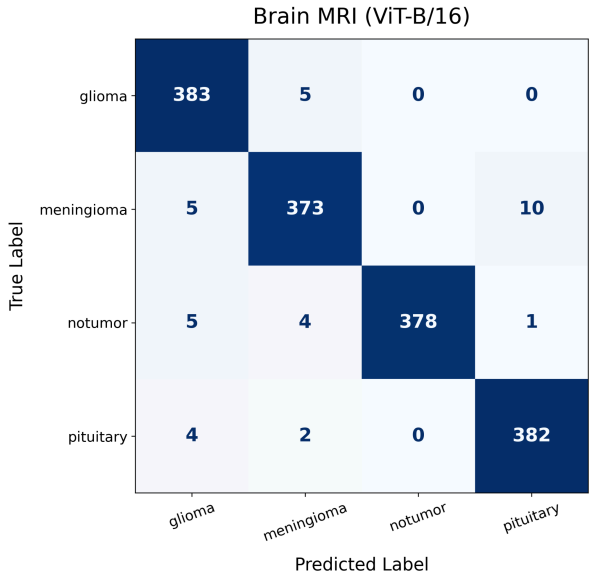


Figure 4.6. Confusion matrix of the best-performing Brain MRI classification model (ViT-B/16).

Figure 4.6 shows that the selected model distinguishes glioma, meningioma, no-tumor, and pituitary classes with very strong overall performance. The limited number of off-diagonal predictions indicates that the model captures the dominant visual patterns of each tumor category effectively, although a small degree of confusion remains among clinically related classes.

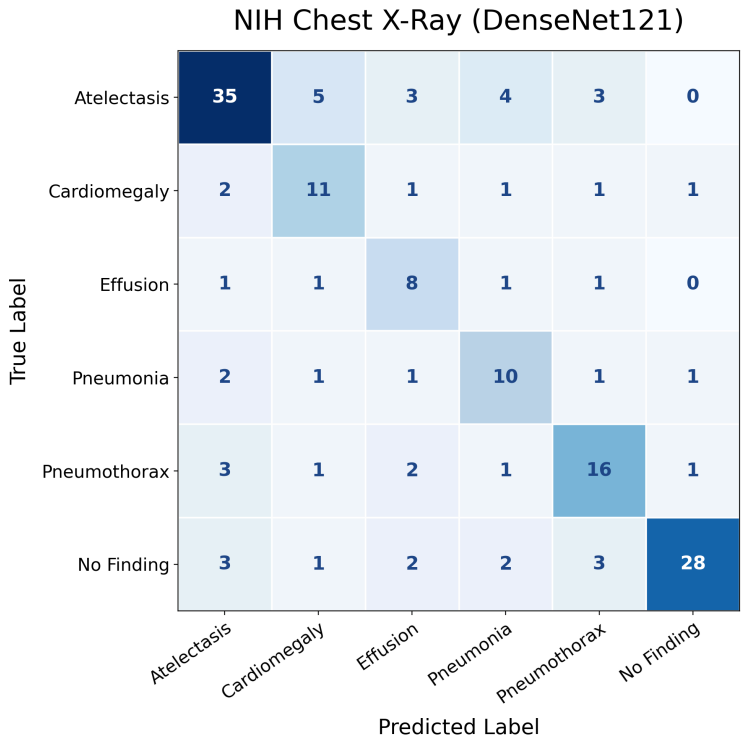


Figure 4.7. Confusion matrix of the best-performing NIH Chest X-ray classification model (DenseNet121).

Figure 4.7 shows that the selected model distinguishes the target thoracic findings with generally strong overall performance, while still exhibiting limited confusion among clinically related chest pathologies. The concentration of predictions along the diagonal indicates that DenseNet121 captures the dominant radiographic patterns effectively; however, the remaining off-diagonal errors suggest that certain findings with overlapping visual characteristics remain more difficult to separate perfectly.

Taken together, these visual analyses strengthen the evaluation by showing not only how accurate the selected models are overall, but also how their errors are distributed across clinically meaningful classes. This makes the integrated AI layer more transparent and helps justify why different architectures were selected for different tasks.

4.3. Additional Experimental Result

In addition to the seven integrated classification tasks, an additional segmentation-oriented experiment was conducted using a U-Net-based model for brain tumor MRI analysis (Ronneberger et al., 2015). Unlike the classification tasks, this study focused on pixel-level localization of abnormal regions rather than assigning a single image-level category. The U-Net-based segmentation model achieved a test Dice score of 0.8023, indicating that the predicted tumor regions showed strong overlap with the ground-truth masks. Although this model was not integrated into the final deployed platform, it represents an important extension of the project because it explores a more advanced form of medical image understanding.

5. Impact of Engineering Solutions

The engineering solutions developed in this project have implications that extend beyond software functionality. Because the system operates in a healthcare-related domain, its value must be evaluated not only in technical terms but also in economic, environmental, and societal contexts. Architectural decisions such as modular service separation, role-based workflow design, centralized inference control, audit logging, and safety-oriented response shaping directly affect deployment cost, resource usage, accessibility, trust, and responsible system behavior.

5.1. Global Impact

The global impact of this project stems from the broader challenge of making healthcare support more accessible, scalable, and responsibly assisted by artificial intelligence across different regions and resource conditions. In many parts of the world, access to medical expertise, diagnostic support, and timely preliminary guidance is unevenly distributed. A web-based, AI-assisted platform such as the one developed in this project has the potential to contribute to more consistent and structured access to early-stage medical support, especially in contexts where specialist availability is limited.

At the same time, the project also reflects global challenges associated with the deployment of AI in healthcare. Differences in language, healthcare infrastructure, regulatory environments, dataset representativeness, and clinical practice standards mean that such systems cannot be assumed to perform equally well in every setting. For this reason, the project's engineering approach emphasized modularity, role-awareness, traceability, and controlled AI behavior rather than universal autonomous decision-making. This is important from a global perspective, because healthcare technologies must be adaptable to different institutional, cultural, and operational contexts.

In addition, the project aligns with a broader international movement toward human-centered and ethically governed AI systems in medicine. Rather than replacing

doctors, the system positions AI as a supportive layer within supervised workflows. In this sense, its global significance lies not only in technical functionality, but also in demonstrating how AI-assisted healthcare platforms can be designed with safety, accountability, and adaptability in mind.

5.2. Economic Impact

From an economic perspective, the system has the potential to reduce time-related and process-related costs in healthcare support workflows. For doctors, AI-assisted image analysis, structured case management, annotation support, and quick access to patient interaction history can reduce repetitive effort and improve workflow efficiency. For patients, an accessible symptom-oriented interface may help users obtain earlier structured guidance before unnecessary escalation, which can reduce avoidable workload in low-risk situations.

The project also demonstrates the economic advantage of combining multiple diagnostic support functions within a single platform rather than maintaining isolated tools. A unified frontend, common backend services, shared authentication infrastructure, and centralized logging and persistence reduce duplication in development and maintenance. In addition, the use of open-source frameworks and modular deployment practices supports lower development-entry cost and easier extensibility.

However, the platform also introduces important cost factors. Training and maintaining multiple AI models, hosting backend services, operating external language-model integration, validating outputs, and monitoring safety-related behavior all require computational and engineering resources. In healthcare-related software, these costs are justified only when the system remains supportive, reliable, and well-governed. Therefore, the project's economic significance lies not in eliminating human expertise, but in improving the efficiency and consistency of expert-supported workflows through responsible AI integration.

5.3. Environmental Impact

The environmental impact of the project is primarily related to computational resource consumption. Training deep learning models for multiple medical imaging tasks and operating inference services require GPU usage, electrical energy, storage, and supporting digital infrastructure. In particular, repeated experimentation during model comparison and optimization can increase the environmental footprint of development.

At the same time, several engineering choices in the project help limit unnecessary resource use. The system uses task-specific inference paths instead of running all models for every request, which reduces avoidable computation during operation. The modular architecture also enables targeted updates, reuse of shared backend components, and more controlled deployment practices, preventing wasteful duplication

of software services. Container-based execution further supports reproducibility and reduces the need for repeated environment reconfiguration.

Even so, the project should not be viewed as environmentally neutral. Future improvements should include more efficient model selection, reduced model size where performance permits, better caching strategies, selective inference execution, and more careful use of high-cost compute resources during retraining. As a result, the environmental relevance of the project lies in balancing the medical value of AI-assisted support against the compute intensity required to develop and operate such systems responsibly.

5.4. Societal Impact

The societal impact of the project is potentially significant because it addresses access, communication, and support in a medically sensitive domain. For patients, the platform can provide earlier and more understandable interaction with health-related information through a controlled conversational interface. For doctors, it can support review workflows, highlight AI-generated findings, and improve traceability through annotations, overrides, and audit records. In this sense, the system may contribute to better access to structured support and more organized clinical interaction.

At the same time, societal benefit depends heavily on safe system boundaries. In healthcare, poorly controlled AI systems can create misinformation, anxiety, overconfidence, or unfair outcomes. For this reason, the platform intentionally avoids presenting AI as an autonomous diagnostic authority. Consent checks, role-based response shaping, disclaimer attachment, moderation, safe fallback behavior, and doctor override mechanisms were incorporated specifically to reduce the risk of harmful misinterpretation.

There are also important societal risks that must be acknowledged. Model performance may vary across populations, datasets, and medical conditions, and any system trained on public or limited datasets may inherit bias or underperform in underrepresented cases. In addition, some users may be tempted to over-rely on automated outputs if the limits of the system are not clearly communicated. Therefore, the societal value of the project depends not only on technical capability, but also on transparency, human oversight, fairness awareness, and continued evaluation in realistic contexts.

6. Contemporary Issues Discussion

The intersection of software engineering and healthcare introduces strict requirements regarding data privacy, system reliability, and the ethical deployment of artificial intelligence. The architectural decisions in this project were driven by the need to address these contemporary challenges directly, ensuring that the system remains both practically viable and safety-aware for healthcare-related use.

- **Data Sovereignty and Patient Privacy:** One of the most critical contemporary issues in healthcare technology is the protection of sensitive personal health information (PHI). Relying on external AI services may introduce privacy, control, and regulatory concerns. In this project, these risks were addressed through a controlled backend-mediated architecture. Medical image models were executed within the system environment, while symptom analysis was performed through an external HPC-hosted language model service integrated behind consent checks, role-based access control, disclaimer enforcement, and safe fallback behavior. This design reduced direct exposure of raw AI services to end users and helped preserve a more privacy-aware and traceable workflow.

- **Ethical AI Implementation in Clinical Decision Support:** The rapid introduction of AI into medicine raises ethical concerns regarding automation, diagnostic bias, and accountability. This project approaches AI not as an autonomous diagnostic entity, but strictly as a clinical decision support system (CDSS) for medical professionals. The vision models and NLP components are integrated to assist physicians by pre-processing medical imagery, extracting relevant information from unstructured patient inputs, and highlighting potential anomalies. By maintaining a "human-in-the-loop" approach, the system reduces the cognitive load on healthcare providers and decreases diagnostic latency, while ensuring that the final medical authority and accountability always remain with the human doctor.

- **High Availability and Observability in Mission-Critical Systems:** In a medical setting, software downtime directly impacts patient care and clinic workflows. Therefore, maintaining high availability and system stability is a primary engineering concern. The project addresses the contemporary demand for fault-tolerant infrastructure through containerization and robust observability. By isolating the frontend, backend, database, and heavy machine learning workloads into dedicated Docker containers, the system prevents process interference (e.g., a heavy NLP inference task crashing the web server). Furthermore, the integration of Prometheus, Grafana, and Loki establishes a comprehensive telemetry stack. This allows system administrators to monitor real-time resource consumption, track API response times, and query application logs centrally, enabling the rapid identification of systemic bottlenecks before they result in critical failures during patient-doctor interactions.

- **Computational Resource Management for AI-Enabled Healthcare Systems:** While local execution of image models improves control over sensitive data, AI-assisted healthcare systems still introduce significant computational and infrastructure challenges, especially when multiple inference workloads and external service dependencies must be managed together. In a clinical setting,

multiple physicians might request medical image processing or NLP inference simultaneously. If not handled correctly, these computationally expensive tasks can block the main application thread, rendering the system unresponsive. This project mitigates this bottleneck through an asynchronous backend architecture. By utilizing FastAPI and ASGI servers (Uvicorn/Gunicorn), the system handles multiple concurrent AI inference requests non-blockingly. Furthermore, the integration of Redis as an in-memory caching layer ensures that repetitive data retrieval or identical model inferences are served directly from the cache, significantly reducing the redundant computational load on the local hardware.

7. Tools and Technologies Used

The development of this project relies on a comprehensive technology stack distributed across frontend interfaces, backend services, data storage, and operational pipelines. The following tools, frameworks, and libraries were specifically selected to support a modular architecture, ensure system reliability, and facilitate efficient data processing throughout the application lifecycle.

1. Frontend Development

- **React:** The core library used for building component-based user interfaces.
- **TypeScript:** Implemented to provide static typing, improving code maintainability and type safety.
- **Vite:** Utilized as the primary build tool and development server.
- **Tailwind CSS:** Applied for utility-first, responsive UI styling.
- **React Router:** Handled client-side routing and navigation across the application.
- **Axios:** Used as the HTTP client to manage asynchronous API requests to the backend.
- **i18next & react-i18next:** Integrated for internationalization (i18n) and localization support.
- **Additional UI Libraries:** React Hook Form was used for form state management, React Markdown for rendering formatted text, and Lucide React for application iconography.

2. Backend Development

- **Python (3.10+):** The primary programming language used for backend logic and data processing.
- **FastAPI:** Utilized as the asynchronous web framework for API development.
- **SQLAlchemy:** Served as the Object-Relational Mapper (ORM) for relational database interactions.
- **Pydantic:** Handled data validation, type checking, and schema management.
- **Uvicorn & Gunicorn:** Deployed as ASGI servers to manage application requests in production.

- **Transformers (Hugging Face):** Integrated for executing machine learning and NLP models within the backend architecture.

3. Database and Data Storage

- **PostgreSQL:** The primary relational database used for persistent data storage, accessed via psycopg2.
- **Redis:** Implemented as an in-memory data store for caching and optimizing backend response times

4. DevOps, Testing, and Monitoring

- **Docker & Docker Compose:** Used to containerize the application, ensuring environment consistency across development and production stages.
- **GitHub Actions & Workflows:** Configured to establish Continuous Integration and Continuous Deployment (CI/CD) pipelines, automating the testing, build, and deployment processes.
- **Prometheus:** Configured for application monitoring and tracking time-series metrics.
- **Grafana:** Integrated as the primary visualization platform to monitor system performance and metrics through centralized dashboards.
- **Loki:** Implemented as a log aggregation system, allowing for efficient storage and querying of application logs.
- **Pytest & Coverage:** Utilized to develop the test suite, covering unit and integration testing to verify code reliability.

8. Use of Resources and Related Work

The development of this project was informed by a combination of recent academic literature, publicly available medical datasets, official technical documentation, and modern software and AI frameworks. These resources supported not only the implementation of the system, but also the definition of its safety boundaries, architectural decisions, and role-specific interaction model. In particular, recent medical AI literature was used to position the project within the broader transition from isolated predictive models toward clinically meaningful, system-level AI support tools (Esteva et al., 2019; Rajpurkar et al., 2022).

From a software engineering perspective, the project relied on widely adopted implementation technologies. The frontend was developed using React and TypeScript to support a role-aware and browser-based user experience, while the backend was implemented with FastAPI to enable modular service design, API-driven communication, and structured separation of concerns. Persistent storage and audit-oriented logging mechanisms were incorporated to support traceability and session continuity. In addition, containerized deployment practices were used to improve reproducibility and

environmental consistency. These technical resources were not only implementation tools, but also architectural enablers, allowing the project to move beyond a model demo and toward an integrated platform.

In the AI domain, the project drew on recent advances in both medical imaging and language-based medical assistance. For image analysis, the design was influenced by literature showing that deep learning has become highly effective across multiple medical imaging domains, while also revealing the importance of validation quality, reporting standards, and task-specific design (Aggarwal et al., 2021; Rajpurkar et al., 2022). The use of architectures such as EfficientNet and Vision Transformer was also supported by broader machine learning research demonstrating strong transferability, scaling behavior, and representational capacity in visual tasks (Tan and Le, 2019; Dosovitskiy et al., 2021). In our own system, these ideas were operationalized through a registry-based multi-model structure rather than a single-model pipeline, allowing different diagnostic tasks to be handled through task-specific inference paths.

The model development process was supported by publicly available benchmark-style datasets covering multiple diagnostic tasks and modalities. These included the NIH Chest X-ray Dataset (ChestX-ray14) for thoracic disease classification (Wang et al., 2017), the BraTS dataset and additional Brain MRI Tumor datasets for brain tumor analysis (Menze et al., 2015; Bakas et al., 2017), retinal OCT datasets for ophthalmological imaging, the BUS-BRA breast ultrasound dataset for breast lesion analysis (Gómez-Flores et al., 2024), the COVID-19 Radiography Database for COVID-related imaging tasks (Chowdhury et al., 2020; Rahman et al., 2021), HAM10000 / ISIC-derived dermoscopic datasets for skin lesion classification (Tschandl et al., 2018; Codella et al., 2019), and fracture detection X-ray datasets, including multi-region fracture collections, for musculoskeletal analysis.

A second major resource domain was recent work on large language models in medicine. Post-2022 literature has increasingly highlighted both the opportunities and the risks of LLM use in healthcare. Reviews have shown that LLMs are promising for patient communication, clinical assistance, and medical text understanding, but also raise concerns regarding hallucination, overconfidence, safety, controllability, and appropriate deployment boundaries (Thirunavukarasu et al., 2023; WHO, 2021). Other recent discussions argue that the future of LLMs in medicine depends not only on performance, but also on transparency, controllability, and alignment with clinical workflows (Riedemann et al., 2024). This directly influenced our project. Rather than exposing raw language model output to end users, the implemented system places the symptom-analysis workflow behind consent checks, role-based response shaping, disclaimer enforcement, and safe fallback behavior. In this sense, our project adopts the LLM not as a stand-alone chatbot, but as one component inside a controlled medical software system.

Related work also shows that many existing systems remain limited in scope. A large portion of medical AI studies continue to focus on a single task, a single modality, or a single benchmark dataset, often emphasizing model accuracy while paying less attention to deployment constraints, user separation, traceability, or human override mechanisms

(Aggarwal et al., 2021; Rajpurkar et al., 2022). Likewise, recent LLM studies in healthcare often focus on question answering, empathy, or clinical reasoning quality, but do not necessarily embed these capabilities into a production-style software architecture with consent handling, moderation, logging, and role-sensitive output control (Ayers et al., 2023; Thirunavukarasu et al., 2023). In contrast, our project was designed at the platform level. It combines doctor-facing image analysis, patient-facing symptom assistance, chat-session continuity, audit logging, doctor annotations and overrides, disclaimer enforcement, and monitoring mechanisms in a single integrated environment.

Therefore, the main distinction of our work is not that it introduces an entirely new individual model architecture, but that it combines established AI capabilities with a structured healthcare-oriented software architecture. The project extends beyond isolated model performance by integrating multiple diagnostic tasks, separating doctor and patient experiences, and embedding safety-oriented engineering controls directly into the system. In this respect, the project is more closely aligned with recent calls for multimodal, controllable, and clinically integrated medical AI systems than with conventional single-task academic prototypes (Acosta et al., 2022; Moor et al., 2023). Overall, the resources and related work used in this project helped shape not only the technical implementation, but also the system's core identity as a role-aware, traceable, and ethically constrained AI-assisted healthcare support platform.

9. Engineering Standards & Design Principles

The design and implementation of the AI Health System were guided by a combination of internationally recognized engineering standards and well-established software design principles. These standards were adopted to ensure that the system is structured, maintainable, scalable, secure, and traceable throughout its lifecycle. The Low-Level Design (LLD) phase explicitly builds upon the High-Level Design (HLD) by refining architectural components into implementable software elements, as required in object-oriented system design methodologies .

Adopted Engineering Standards

The project follows several widely accepted standards to ensure consistency, clarity, and professional software development practices:

- **IEEE 830-1998 (Software Requirements Specification)**
  This standard guided the documentation of system requirements, including functional and non-functional requirements, constraints, and system assumptions. Requirements were clearly defined and later traced into design components.

- **IEEE 1016-2009 (Software Design Description)**
  The system design is documented according to IEEE 1016 principles, including architectural decomposition, subsystem responsibilities, interface definitions, workflows, and error handling strategies. Both HLD and LLD documents reflect

this structured design approach.

- **UML 2.x Modeling Principles**
  UML diagrams (class diagrams, sequence diagrams, activity diagrams, and use case diagrams) were used extensively to represent both structural and behavioral aspects of the system. UML serves as the backbone of the LLD, enabling clear communication of system components and interactions.

- **ISO/IEC 25010 (Software Quality Model)**
  The system design incorporates key quality attributes such as:
  - **Security** (access control, audit logging)
  - **Reliability** (controlled workflows, error handling)
  - **Usability** (role-based interfaces for doctors and patients)
  - **Maintainability** (modular architecture)
  - **Performance** (response time constraints)

- **PEP 8 (Python Coding Standards)**
  Backend implementation follows Python best practices, ensuring readable, modular, and maintainable code structure with consistent naming conventions and layered architecture.

- **GDPR and KVKK Compliance Principles**
  Privacy-aware design was integrated through:
  - Consent management
  - Role-based access control (RBAC)
  - Data minimization and anonymization
  - Audit logging for traceability
    While GDPR principles are largely followed, KVKK compliance is considered partially satisfied due to the absence of formal legal validation.

- **HIPAA (Reference Only)**
  HIPAA was considered as a conceptual guideline for healthcare data handling; however, since the system is not a certified clinical product, full compliance was not required.

Core Design Principles

In addition to formal standards, the system was developed using fundamental software engineering design principles:

Modularity and Separation of Concerns

The system is divided into independent subsystems such as:

- User Management
- Image Processing
- Symptom Analysis
- Monitoring & Logging
- Feedback & Moderation

Each subsystem has clearly defined responsibilities, improving maintainability and scalability.

Layered Architecture

The backend follows a structured layered design:

- **Controller Layer** → Handles requests and API endpoints
- **Service Layer** → Contains business logic
- **Model Layer** → Defines data structures
- **Repository Layer** → Manages data persistence

This separation ensures loose coupling and easier testing.

Traceability (HLD → LLD Mapping)

Each high-level module defined in the HLD is systematically decomposed into LLD components (classes, methods, and workflows). This ensures that:

- Every requirement is implemented
- Design decisions are traceable
- System evolution remains consistent

This mapping is a key requirement of LLD methodology.

Scalability and Extensibility

The system is designed to support:

- Multiple AI models (ResNet50, DenseNet121, EfficientNet-B4, ViT)
- Multiple datasets and diagnostic tasks
  New models or modules can be integrated without affecting existing components.

Security by Design

Security is integrated at the design level rather than added later:

- Authentication and authorization mechanisms

- Role-based data access
- Audit logging and monitoring
- Controlled AI output for patients

Fail-Safe and Controlled AI Behavior

Given the healthcare context, the system enforces:

- Mandatory disclaimers for patient outputs
- Moderation pipelines for AI responses
- Doctor override mechanisms

This ensures that the system remains a decision-support tool, not a diagnostic system.

Performance Awareness

Design decisions consider system constraints:

- Image analysis response target: ≤ 10 seconds
- Symptom analysis response target: ≤ 5 seconds
- Support for concurrent users

Design Trade-offs

Due to real-world constraints, several trade-offs were made:

- **Accuracy vs. Efficiency**
  Instead of optimizing all models equally, only the best-performing models per task were integrated.
- **Security vs. Usability**
  Additional steps such as authentication and controlled outputs reduce risk but slightly increase interaction complexity.
- **Generality vs. Practical Feasibility**
  The system supports multiple medical tasks, but not at full clinical generalization level due to dataset and time constraints.
- **External Dependency vs. Functionality**
  The symptom analysis module depends on an external LLM service; this is mitigated through application-level safeguards.

Overall, the system adheres to established engineering standards while applying robust software design principles. The result is a modular, traceable, secure, and extensible AI-based healthcare platform. Although the system is not a production-grade clinical tool, it represents a well-structured engineering prototype that balances innovation, feasibility, and responsible design.

10. New Tools & Technologies Used

This project incorporated several contemporary AI technologies that extend beyond conventional full-stack software development. While the overall system was built using standard web and backend engineering tools, the most notable new technologies used during the course were recent vision architectures, medical image segmentation methods, and controlled large language model integration for multilingual symptom analysis.

One of the most important new technologies explored in the project was the use of Vision Transformer (ViT-B/16) models for medical image classification. Unlike traditional convolution-based approaches, ViT applies the transformer paradigm to image patches and enables stronger global-context modeling. In this project, ViT-based models were trained and evaluated for multiple medical imaging tasks, including brain MRI classification, retinal OCT analysis, breast ultrasound classification, fracture detection, COVID radiography analysis, and skin lesion classification. Their use provided practical experience with a modern architecture family that has become increasingly influential in recent computer vision research.

Another important new technology direction was medical image segmentation using a U-Net-based architecture. Although the U-Net experiment was not integrated into the final deployed platform, it introduced the project to pixel-level medical image understanding rather than only image-level classification. This was particularly valuable in the context of brain tumor MRI analysis, where localization of abnormal tissue is often more informative than a single class label.

A further major new technology area was controlled large language model integration through an external HPC-hosted service. The symptom-analysis pipeline was built around MedGemma-27B-IT rather than a rule-based chatbot or template-based text engine. In addition, because the patient-facing workflow needed to support Turkish interaction, the system incorporated machine translation support using Helsinki-NLP models for Turkish-to-English and English-to-Turkish conversion. This made it possible to process symptom descriptions through an English-centered medical language model while still returning understandable Turkish responses to users. As a result, the project combined modern LLM-based medical text generation with multilingual interaction support and application-level safeguards such as consent control, disclaimer enforcement, moderation, and role-aware output shaping.

Overall, the genuinely new technologies used in this project were not the standard software frameworks supporting the platform, but the contemporary AI capabilities embedded into it. These included transformer-based vision models such as ViT-B/16, segmentation-oriented medical imaging with U-Net, and multilingual LLM-supported symptom interaction through MedGemma-27B-IT and Helsinki-NLP translation models. Together, these technologies gave the project a stronger connection to current developments in artificial intelligence and AI-assisted healthcare systems.

11. Discussion, Evaluation & Future Work

This project demonstrates that an AI-assisted healthcare support platform can be implemented as a coherent software system rather than as a collection of isolated model demonstrations. The final platform brings together role-based web interfaces, backend-controlled workflows, task-specific image inference, symptom-oriented language analysis, audit logging, monitoring, and safety-oriented response handling within a unified architecture. From an engineering standpoint, the main contribution of the project is the structured integration of artificial intelligence into a traceable, modular, and user-aware healthcare support environment.

The overall evaluation of the project is positive. The completed tests confirmed that the main system requirements were met across authentication, authorization, consent enforcement, symptom analysis, doctor-facing image workflows, localization, logging, and monitoring behaviors. The architecture also proved flexible enough to support multiple medical imaging tasks through a registry-based model structure while maintaining controlled patient-facing interaction through application-level safeguards. This indicates that the system achieved its primary goal of using AI as supportive decision assistance rather than as an uncontrolled autonomous actor.

At the same time, the project has clear limitations. First, the quality of AI outputs remains dependent on the datasets used during training and evaluation, which means that real-world generalization cannot be assumed from development-time results alone. Second, model performance is not uniform across all medical tasks; some tasks produce stronger results than others, showing that deployment readiness must be assessed individually rather than globally. Third, dependence on external inference services introduces latency and reliability considerations that can affect real-time user experience. Finally, although the platform includes safety controls and traceability mechanisms, it has not been validated in real clinical environments, and therefore it should be considered a decision-support prototype rather than a clinically deployed product.

These limitations lead directly to several future-work directions. A first priority is broader and more diverse dataset coverage, including stronger validation across class imbalance, domain shift, and underrepresented medical cases. A second priority is deeper model evaluation through calibration analysis, uncertainty estimation, fairness-oriented assessment, and continued false-negative/false-positive monitoring for clinically sensitive classes. A third priority is operational maturity, including larger-scale load testing, stronger failure recovery, reduced inference latency, and more advanced monitoring of external service dependencies. Additional development directions include integration of richer explainability mechanisms, hospital-oriented customization, stronger multimodal fusion, and support for segmentation-based workflows such as the brain tumor segmentation direction explored during model development. A further future-work direction is hospital-specific model adaptation using institutionally collected and properly governed local datasets. Retraining or fine-tuning the system on local imaging

distributions, reporting styles, and patient populations could improve robustness against domain shift and make the platform more clinically relevant within specific healthcare environments. This would be particularly valuable for aligning the system with local workflow patterns, equipment characteristics, and population-specific diagnostic distributions.

Overall, the project provides a strong foundation for future extension. Its most important outcome is not only that multiple AI capabilities were implemented, but that they were organized within a software architecture that emphasizes human oversight, controlled interaction, and engineering accountability. Future improvements can therefore build on an already structured system and move it closer to robust, real-world healthcare support use.

12. Conclusion

This project resulted in the development of a web-based, AI-assisted medical decision-support platform designed for both doctors and patients. By combining medical image analysis, symptom-based language analysis, role-based access control, and traceable interaction workflows, the system aimed to provide a practical and responsible use of artificial intelligence within a healthcare support context. Rather than treating AI as an autonomous diagnostic authority, the project consistently positioned it as a supportive component within human-centered clinical and patient-facing workflows.

From an engineering perspective, the project demonstrates the successful integration of multiple software and AI components into a unified platform. The final system includes a browser-based frontend, a modular FastAPI backend, database-backed persistence, multiple task-specific image inference models, and an external language-model-based symptom analysis service. In addition to core functionality, the system also incorporates auditability, logging, monitoring, safety-oriented output handling, and role-specific behavior. These characteristics were essential in shaping the platform as more than a simple demonstration system, moving it toward a structured and accountable decision-support application.

The project also highlights the importance of balancing technical capability with ethical and operational constraints. In medical contexts, system usefulness cannot be evaluated only in terms of model output; it must also be assessed in terms of safety, interpretability, access control, controlled failure behavior, and the ability to preserve human oversight. For this reason, consent enforcement, disclaimer attachment, moderated patient-facing outputs, doctor override mechanisms, and audit logging were treated as fundamental design and implementation priorities.

At the same time, the project revealed several practical challenges associated with AI-assisted healthcare systems. External inference dependencies, latency variability, infrastructure constraints, and the gap between ideal architectural design and real

implementation conditions all influenced the system's behavior during development. These challenges are not weaknesses unique to this project; rather, they reflect the broader realities of deploying intelligent systems in sensitive and high-responsibility domains. Addressing them required not only technical integration work but also careful architectural and safety-oriented decision making.

Overall, the AI Health System for Doctors and Patients demonstrates that a multi-role, AI-assisted healthcare platform can be designed and implemented in a way that is technically functional, ethically aware, and architecturally structured. The project establishes a strong foundation for future improvements in model quality, system robustness, deployment maturity, and broader clinical applicability. More importantly, it shows that meaningful engineering value in healthcare AI lies not only in predictive performance, but also in the responsible design of the software system that surrounds and governs that intelligence.

13. References

Acosta, J. N., Falcone, G. J., Rajpurkar, P., et al. (2022). Multimodal biomedical AI. Nature Medicine, 28, 1773-1784. https://www.nature.com/articles/s41591-022-01981-2

Aggarwal, R., Sounderajah, V., Martin, G., et al. (2021). Diagnostic accuracy of deep learning in medical imaging: a systematic review and meta-analysis. npj Digital Medicine, 4, 65. https://www.nature.com/articles/s41746-021-00438-z

Ayers, J. W., Poliak, A., Dredze, M., et al. (2023). Comparing Physician and Artificial Intelligence Chatbot Responses to Patient Questions Posted to a Public Social Media Forum. JAMA Internal Medicine, 183(6), 589-596. https://jamanetwork.com/journals/jamainternalmedicine/fullarticle/2804309

Bakas, S., Akbari, H., Sotiras, A., et al. (2017). Advancing The Cancer Genome Atlas glioma MRI collections with expert segmentation labels and radiomic features. Scientific Data, 4, 170117. https://doi.org/10.1038/sdata.2017.117

Chowdhury, M. E. H., Rahman, T., Khandakar, A., et al. (2020). Can AI help in screening Viral and COVID-19 pneumonia? IEEE Access, 8, 132665-132676. https://ieeexplore.ieee.org/document/9144185

Codella, N. C. F., Rotemberg, V., Tschandl, P., et al. (2019). Skin Lesion Analysis Toward Melanoma Detection 2018: A Challenge Hosted by the International Skin Imaging Collaboration (ISIC). arXiv preprint arXiv:1902.03368. https://arxiv.org/abs/1902.03368

Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021. https://openreview.net/forum?id=YicbFdNTTy

Esteva, A., Robicquet, A., Ramsundar, B., et al. (2019). A guide to deep learning in healthcare. Nature Medicine, 25, 24-29. https://www.nature.com/articles/s41591-018-0316-z

European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation). https://eur-lex.europa.eu/legal-content/en/ALL/?uri=CELEX%3A32016R0679

Gómez-Flores, W., Gregorio-Calas, M. J., & de Albuquerque Pereira, W. C. (2024). BUS-BRA: A Breast Ultrasound Dataset for Assessing Computer-aided Diagnosis Systems. Medical Physics, 51, 3110-3123. https://doi.org/10.1002/mp.16812

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778. https://doi.org/10.1109/CVPR.2016.90

Helsinki-NLP. (n.d.). opus-mt-tr-en. Hugging Face. https://huggingface.co/Helsinki-NLP/opus-mt-tr-en

Helsinki-NLP. (n.d.). opus-mt-tc-big-en-tr. Hugging Face. https://huggingface.co/Helsinki-NLP/opus-mt-tc-big-en-tr

Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 4700-4708. https://doi.org/10.1109/CVPR.2017.243

Kişisel Verileri Koruma Kurumu. (2016). Personal Data Protection Law No. 6698. https://www.kvkk.gov.tr/Icerik/6649/Personal-Data-Protection-Law

Menze, B. H., Jakab, A., Bauer, S., et al. (2015). The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS). IEEE Transactions on Medical Imaging, 34(10), 1993-2024. https://doi.org/10.1109/TMI.2014.2377694

Moor, M., Banerjee, O., Abad, Z. S. H., et al. (2023). Foundation models for generalist medical artificial intelligence. Nature, 616, 259-265. https://www.nature.com/articles/s41586-023-05881-4

Nickparvar, M. (2024). Brain Tumor MRI Dataset [Data set]. Kaggle. https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

Rahman, T., Khandakar, A., Qiblawey, Y., et al. (2021). Exploring the effect of image enhancement techniques on COVID-19 detection using chest X-ray images. Computers in Biology and Medicine, 132, 104319. https://doi.org/10.1016/j.compbiomed.2021.104319

Rajpurkar, P., Chen, E., Banerjee, O., & Topol, E. J. (2022). AI in health and medicine. Nature Medicine, 28, 31-38. https://www.nature.com/articles/s41591-021-01614-0

Riedemann, L., Labonne, M., & Gilbert, S. (2024). The path forward for large language models in medicine is open. npj Digital Medicine, 7, 339. https://www.nature.com/articles/s41746-024-01344-w

Rodrigo, M. (2023). Bone Fracture Multi-Region X-ray Data [Data set]. Kaggle. https://www.kaggle.com/datasets/bmadushanirodrigo/fracture-multi-region-x-ray-data

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In Medical Image Computing and Computer-Assisted Intervention (MICCAI), LNCS 9351, 234-241. https://lmbweb.informatik.uni-freiburg.de/Publications/2015/RFB15a

Sairam, V. A. (2023). Bone Fracture Detection Using X-rays [Data set]. Kaggle. https://www.kaggle.com/datasets/vuppalaadithyasairam/bone-fracture-detection-using-xrays

Sellergren, A., Kazemzadeh, S., Jaroensri, T., et al. (2025). MedGemma Technical Report. arXiv preprint arXiv:2507.05201. https://doi.org/10.48550/arXiv.2507.05201

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Proceedings of the 36th International Conference on Machine Learning. https://proceedings.mlr.press/v97/tan19a.html

Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., et al. (2023). Large language models in medicine. Nature Medicine, 29, 1930-1940. https://www.nature.com/articles/s41591-023-02448-8

Tiedemann, J. (2020). The Tatoeba Translation Challenge: Realistic Data Sets for Low Resource and Multilingual MT. In Proceedings of the Fifth Conference on Machine Translation, 1174-1182. https://doi.org/10.18653/v1/2020.wmt-1.139

Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data, 5, 180161. https://doi.org/10.1038/sdata.2018.161

U.S. Department of Health and Human Services. (n.d.). HIPAA for Professionals. https://www.hhs.gov/hipaa/for-professionals/index.html

Wang, X., Peng, Y., Lu, L., Lu, Z., Bagheri, M., & Summers, R. M. (2017). ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2097-2106. https://openaccess.thecvf.com/content_cvpr_2017/html/Wang_ChestX-Ray8_Hospital-Scale_Chest_CVPR_2017_paper.html

World Health Organization. (2021). Ethics and governance of artificial intelligence for health: WHO guidance. https://iris.who.int/handle/10665/341996

World Health Organization. (2024). Ethics and governance of artificial intelligence for health: guidance on large multi-modal models. https://iris.who.int/handle/10665/375579